

Homework 3: Value-Based RL and Soft Actor-Critic

Student ID: 112550081

Problem 1 (Soft Policy Improvement for Regularized MDPs) — 20 pts

We want to solve the constrained optimization

$$\max_{\pi(\cdot|s)} \sum_{a \in \mathcal{A}} \left(\pi(a|s) Q_{\Omega}^{\pi_k}(s, a) - \pi(a|s) \log \pi(a|s) \right), \quad \text{s.t.} \sum_{a \in \mathcal{A}} \pi(a|s) - 1 = 0.$$

Introduce a Lagrange multiplier $\mu \in \mathbb{R}$ for the simplex constraint. The Lagrangian is

$$L(\pi) = \sum_{a \in \mathcal{A}} \left(\pi(a|s) Q_{\Omega}^{\pi_k}(s, a) - \pi(a|s) \log \pi(a|s) \right) - \mu \left(\sum_{a \in \mathcal{A}} \pi(a|s) - 1 \right).$$

Step 1 — Take the partial derivative w.r.t. each $\pi(a|s)$. Using

$$\frac{d}{dx} (x \log x) = \log x + 1:$$

$$\frac{\partial L(\pi)}{\partial \pi(a|s)} = Q_{\Omega}^{\pi_k}(s, a) - (\log \pi(a|s) + 1) - \mu.$$

Step 2 — Set the derivative to zero. The stationarity condition

$\partial L / \partial \pi(a|s) = 0$ gives

$$\log \pi(a|s) = Q_{\Omega}^{\pi_k}(s, a) - 1 - \mu \implies \pi(a|s) = \exp(Q_{\Omega}^{\pi_k}(s, a) - 1 - \mu).$$

Step 3 — Enforce the simplex constraint. Since $\sum_a \pi(a|s) = 1$,

$$\exp(-1 - \mu) \sum_{a \in \mathcal{A}} \exp(Q_{\Omega}^{\pi_k}(s, a)) = 1 \implies \exp(-1 - \mu) = \frac{1}{\sum_{a' \in \mathcal{A}} \exp(Q_{\Omega}^{\pi_k}(s, a'))}$$

Step 4 — Substitute back. We obtain the closed-form optimum (a *softmax* / Boltzmann policy):

$$\pi_{k+1}(a|s) = \frac{\exp(Q_{\Omega}^{\pi_k}(s, a))}{\sum_{a' \in \mathcal{A}} \exp(Q_{\Omega}^{\pi_k}(s, a'))}.$$

The second-order test confirms a maximum: $\partial^2 L / \partial \pi(a|s)^2 = -1/\pi(a|s) < 0$, so L is strictly concave and the stationary point is the unique global maximizer. ■

Problem 2(a) — Actor Network of SAC (10 pts)

(i) Why output the mean and the log-standard-deviation?

SAC parameterizes a **stochastic Gaussian policy** $\pi_{\phi}(a|s) = \mathcal{N}(\mu_{\phi}(s), \sigma_{\phi}(s)^2)$ for *continuous* actions, so the network must output both location (μ) and scale (σ) parameters.

We predict $\log \sigma$ rather than σ directly for two reasons:

1. **Positivity is automatic.** $\sigma = \exp(\log \sigma) > 0$ for any real-valued network output, so we do not need an extra activation (e.g. softplus or ReLU) that could cause vanishing gradients near zero.
2. **Numerical stability and wider dynamic range.** Standard deviations span many orders of magnitude (e.g. 10^{-9} to 10^2). Operating in log-space yields better-conditioned gradients, and clamping $\log \sigma \in [-20, 2]$ (as in `sac_starter.py` lines 38–39, 63) prevents the distribution from collapsing to a delta function or exploding.

(ii) How is the reparameterization trick implemented?

A naïve sampling $a \sim \mathcal{N}(\mu_{\phi}, \sigma_{\phi}^2)$ would be **non-differentiable** with respect to ϕ , blocking gradients from the policy-loss back to the actor weights. The reparameterization trick rewrites the sample as a deterministic transformation of an independent noise variable:

$$a = \mu_{\phi}(s) + \sigma_{\phi}(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Now a is a differentiable function of ϕ , and the gradient ∇_{ϕ} can be propagated through both μ_{ϕ} and σ_{ϕ} .

In `sac_starter.py` lines 67–68 this is realized by

```
dist = Normal(mu, std)
z = dist.rsample()          # rsample() uses the reparam. trick
```

```
# equivalent to: z = mu + std *  
torch.randn_like(mu)
```

PyTorch's `rsample()` (as opposed to `sample()`) returns a *reparameterized* sample that preserves the computation graph, allowing gradient flow.

(iii) Why is the log-probability adjusted (lines 72–74)?

The action that is actually applied to the environment is squashed to $(-1, 1)$ via the `tanh` transform:

$$a = \tanh(z), \quad z \sim \mathcal{N}(\mu, \sigma^2).$$

Because `tanh` is an **invertible change of variables**, the density of the transformed variable must be corrected by the Jacobian (Appendix C of the original SAC paper):

$$\log \pi(a|s) = \log \mu(z|s) - \sum_{i=1}^{|\mathcal{A}|} \log(1 - \tanh(z_i)^2),$$

since $\frac{da_i}{dz_i} = 1 - \tanh(z_i)^2$. The starter code

```
action = z.tanh()  
log_prob = dist.log_prob(z) - torch.log(1 - action.pow(2) + 1e-7)  
log_prob = log_prob.sum(-1, keepdim=True)
```

implements exactly this correction. Without it, the entropy term used in the SAC objective would be **biased** (computed in z -space instead of a -space) and the actor would learn a wrong policy. The $+ 1e-7$ term prevents $\log(0)$ when actions saturate near ± 1 , and `.sum(-1)` aggregates over the action dimensions because each is conditionally independent.

Problem 2(b) — Soft Q Network of SAC (10 pts)

(i) Why two Q networks (the “twin Q” / clipped-double-Q trick)?

In standard Q -learning the target $y = r + \gamma \max_{a'} Q(s', a')$ contains a maximization over a noisy estimator, which produces a **positive (overestimation) bias** that compounds through bootstrapping and destabilizes training (Hasselt 2010; Fujimoto et al., *TD3*, 2018).

SAC adopts TD3's **Clipped Double-Q** remedy: maintain two independently-initialized critics $Q_{\theta_1}, Q_{\theta_2}$ and use their *minimum* whenever a Q-estimate is required as a bootstrapping target (e.g. in the V-loss or the actor loss):

$$\hat{Q}(s, a) = \min (Q_{\theta_1}(s, a), Q_{\theta_2}(s, a)).$$

Taking the minimum is a pessimistic correction that systematically counteracts the upward bias of the max-operator, yielding more accurate value estimates and substantially lowering the variance of the policy gradient.

(ii) Loss function for the two soft Q networks

With the value-network target $V_{\bar{\psi}}$, the TD target is

$$y_i = r_i + \gamma (1 - d_i) V_{\bar{\psi}}(s'_i),$$

and each critic is trained with the mean-squared Bellman error:

$$\mathcal{L}_{Q_j}(\theta_j) = \mathbb{E}_{(s_i, a_i, r_i, s'_i, d_i) \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\theta_j}(s_i, a_i) - y_i)^2 \right], \quad j = 1, 2.$$

In mini-batch form (size N):

$$\mathcal{L}_{Q_j} = \frac{1}{N} \sum_{i=1}^N \left(Q_{\theta_j}(s_i, a_i) - [r_i + \gamma (1 - d_i) V_{\bar{\psi}}(s'_i)] \right)^2.$$

The target y_i is *detached* from the computation graph (no gradient through $V_{\bar{\psi}}$).

Problem 2(c) — Alpha Network of SAC (10 pts)

The enhanced SAC paper (Haarnoja et al., 2018; Sec. 5 of <https://arxiv.org/abs/1812.05905>) reformulates the maximum-entropy RL problem as a *constrained* optimization in which the average entropy of the policy must exceed a target $\mathcal{H}_0$ (chosen heuristically as $-\dim(\mathcal{A})$, see line 217 of the starter):

$$\max_{\pi_{0:T}} \mathbb{E} \left[\sum_t r_t \right] \quad \text{s.t.} \quad \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [-\log \pi(a_t | s_t)] \geq \mathcal{H}_0 \quad \forall t.$$

Forming the Lagrangian with multiplier $\alpha \geq 0$ and applying dual descent, the *dual* problem (with the inner-policy optimum frozen) reduces, for each step, to

$$\min_{\alpha \geq 0} \mathbb{E}_{a_t \sim \pi_t^*} \left[-\alpha \log \pi_t^*(a_t | s_t) - \alpha \mathcal{H}_0 \right] = \min_{\alpha \geq 0} \mathbb{E}_{a_t \sim \pi_t^*} \left[-\alpha \left(\log \pi_t^*(a_t | s_t) + \mathcal{H}_0 \right) \right].$$

Optimizing in **log-space** ($\alpha = \exp(\log \alpha)$) automatically enforces positivity and improves numerical stability. With a minibatch estimate, the loss in line 286 of `sac_starter.py` is

$$\mathcal{L}(\alpha) = \frac{1}{N} \sum_i \left[-\exp(\log \alpha) \cdot \left(\log \pi_\phi(a_i | s_i) + \mathcal{H}_0 \right) \right],$$

with $\log \pi_\phi(a_i | s_i)$ **detached** (the multiplier only adapts α , not the policy).

Intuition of the update. Taking the gradient w.r.t. $\log \alpha$:

- If the current entropy $-\mathbb{E}[\log \pi] < \mathcal{H}_0$ (policy too deterministic), then $\log \pi + \mathcal{H}_0 > 0$, and the gradient *increases* $\alpha \rightarrow$ stronger entropy bonus $\rightarrow$ more exploration.
- If the entropy is already above the target ($\log \pi + \mathcal{H}_0 < 0$), the gradient *decreases* $\alpha \rightarrow$ less entropy weight $\rightarrow$ more greedy exploitation.

Thus α self-adjusts to keep the policy entropy near the target, removing the need to hand-tune one of SAC's most sensitive hyperparameters.
